



Stop Sequences

Stop Sequences

1) What they are (and why they matter)

Stop sequences are strings that tell the model, "**stop generating now.**" They give you precise, *semantic* end-points—better than arbitrary cutoffs from `max_tokens`. Typical uses:

- **Structured output:** end exactly after JSON / a table / a section.
- **Agents/ReAct:** stop after `FINAL_ANSWER:` or before tool output bleeds into the reply.
- **UI safety:** prevent extra chatter, signatures, or trailing whitespace.

| Mental model: `max_tokens` is a backstop; stop is the finish line.

2) How they work (practically)

- You provide **one or more** stop strings (e.g., `["__END__", "###"]`).
- Generation halts **as soon as** the model emits a sequence of tokens that **matches** any stop string.
- **Return value:** Most APIs omit the stop string from the returned text, but you should **always trim/strip** it in post-processing to be safe.

Important nuances

- Matching is **exact** (string-level), so whitespace and casing matter.
 - Unicode/whitespace variants can cause misses; if needed, include **multiple canonical variants** in the stop list.
 - If `max_tokens` is reached **before** a stop appears, the output ends anyway (risk of truncation).
-

3) Choosing good stop sequences

Do

- Use **unlikely sentinels**: `"<<END>>"`, `"__END__"`, `"<E0C_9a73>"` (GUID-like).
- Fence sections with **paired tags**: `<<PART 1>> ... <<END PART 1>>`.
- Keep them **short** and **unique** (avoid collisions in normal prose).

Avoid

- Common punctuation (`"."`, `"\n"`) unless your content truly can end there.
 - Strings likely to appear in user data (e.g., `"END"` inside code blocks).
 - Invisible characters you can't easily debug.
-

4) Patterns you'll use in production

4.1 Single-section prose with crisp end

Prompt snippet

```
Write two paragraphs explaining diffusion models.  
Stop after the line:  
__END__
```

API

- `stop: ["__END__"]`
 - Post-process: strip trailing newline.
-

4.2 Multi-part responses (bounded sections)

Prompt snippet

```
Produce two parts.  
  
=== PART 1: BULLETS ===  
- 5 bullets max about benefits  
=== END PART 1 ===  
  
=== PART 2: SUMMARY ===
```

```
Write 120–150 words.  
=== END PART 2 ===
```

API

- `stop: ["=== END PART 2 ==="]`
- Generate all at once **or** call twice (once per part) to simplify bounds.

4.3 JSON contracts (when you lack JSON/grammar mode)

Prompt snippet

```
Return ONLY valid JSON. Do not add commentary.  
Stop after the line:  
<<<END_JSON>>>
```

Schema:

```
{ "title": "string", "tags": "string[]" }
```

JSON:

API

- `stop: ["<<<END_JSON>>>"]`
- Validation: parse + JSON Schema → reject/repair if invalid.

4.4 ReAct / tool-using agents

Prompt snippet

```
You may think briefly, then answer.  
When you are ready to answer the user, write:  
FINAL_ANSWER: <your answer>
```

Do not write anything after FINAL_ANSWER.

API

- `stop: ["FINAL_ANSWER:"]` (or stop **after** that line if you want just the answer body)

- Split “reasoning/tool calls” vs “final answer” by using **different stops** per phase.

5) Interactions with other controls

- `max_tokens` : still set a cap; treat it as a **failsafe** if the stop is never emitted.
- **Temperature / top-p / top-k**: don’t affect stop behavior, but higher randomness can **delay** hitting the stop (allocate buffer).
- **Penalties**: neutral; they may change phrasing and slightly affect when the stop appears.

6) Testing & hardening

- **Golden tests**: fixed prompts → assert that output **ends at** your stop sequence and that **no extra text** follows.
- **Adversarial inputs**: include user data that *mentions* your sentinel; ensure you still end correctly (or change sentinel).
- **Variant coverage**: if the model might add whitespace, include variants like `"__END__"` , `"\n__END__"` .
- **Telemetry**: log *why* a call stopped (which stop was hit vs. max-tokens) to spot truncation issues early.

7) Troubleshooting

Symptom	Likely Cause	Fix
Model runs past intended end	Stop string too common / never matched	Use a rarer sentinel; shorten/normalize; add multiple variants
Output cut mid-JSON	Hit <code>max_tokens</code> before stop	Increase cap; reduce verbosity; split into sections
Stop appears <i>inside</i> data	Sentinel collides with content (e.g., code)	Pick a GUID-like sentinel; escape or base64 user blobs
Extra whitespace or trailing junk	Model added spaces/newlines after stop	Trim post-generation; add <code>stop</code> variants with whitespace

Symptom	Likely Cause	Fix
Inconsistent stopping in streaming	Partial tokenization timing	Buffer and scan stream for stop string before emitting

8) Copy-paste snippets

Minimal sentinel

Stop after the line:
<<END>>

API: `stop: ["<<END>>"]`

Two-phase output

First, draft a bullet outline.
Then write the final answer starting with:
FINAL_ANSWER:
Stop after FINAL_ANSWER: content.

Safe JSON without JSON mode

Return ONLY valid JSON. No preamble.
End with:
###END###

{ "title": "...", "items": ["..."] }

API: `stop: ["###END###"]`

9) Key takeaways

- **Stop sequences end output at logical boundaries**, not arbitrary length limits.
- Choose **rare, explicit sentinels**, and include **variants** if whitespace may vary.
- Keep `max_tokens` as a **backup**, but rely on **stops** for clean, bounded, parseable results—especially in **structured outputs** and **ReAct** workflows.